

مراجعة المشروع الشاملة – تقرير الأخطاء

شرح تفصيلي لكل مشكلة لقتها في مشروع ThreadMarket

الخلاصة الأولية: المشروع يبني ويشغل (TypeScript + Prisma تمام)، لكن في **ناسخ من الأخطاء** لازم تتصلّح. أشهرها: موديول الـ Category **فاضية خالص**، وأقسام من الـ Auth **مكسورة** لأن الـ validators مش متفقين مع الـ controllers.

0. إيه اللي أنا فحصته؟

1. شغّلت `npm run build` – تأكد إن الكود بيتجمع من غير أخطاء TypeScript.
2. شغّلت `npx prisma validate` – تأكد إن السكيما سليمة.
3. قريت كل الملفات: `auth` ، `user` ، `brand` ، `Category` ، `middleware` ، والـ `utils`.
4. قارنت إيه المطلوب في التاسك وإيه الفعلي في الكود.

1. الوضع الحالي – الشغال تمام

- `npm run build` – يعدي من غير أخطاء.
- `prisma validate` – السكيما سليمة، والتعديلات (`brandId + imageUrl + العلاقة العكسية`) اتحفظت فعلاً في الداتابيز.
- موديول `user` (جلب/تعديل البروفايل) – سليم ومنفذ صح.
- موديول `brand` (حالة التحقق) – سليم.
- الـ `middleware: authenticate` (يقطع التوكن) و `authorize` (يحدد الأدوار) – سليمين.

2. المشكلة رقم 1 – موديول Category فاضية خالص

عملت فولدر Category والملفات جواه فاضية تمامًا (0 سطر):

```
src/modules/Category/  
├─ category.controller.ts ← فاضي  
├─ category.service.ts ← فاضي  
├─ category.model.ts ← فاضي  
├─ category.routes.ts ← فاضي  
└─ category.validators.ts ← فاضي
```

ده جوهر التاسك (POST /brands/:brandId/categories) ومفيش فيه حاجة متعملة.
الملخص في ملف create-category.pdf بيوصف إزاي تكتب كل ملف.

3. مشكلة رقم 2 – ال Auth مكسور

إيه معنى "ال validator مش مطابق لل controller"؟ كل ملف بيشتغل لوحده صح، بس
بيتكلموا بأسماء مختلفة لنفس الحاجة. مثلاً:

ال Endpoint	ال validator بيطلب حقل	الكود بيقرأ حقل	النتيجة
/login	email	identifier	هيقف 401/500 دايمًا
/register	name	username	اليوزر هيتعمل بـ undefined
/register- verify-otp	email	token	عمره ما يعدي ال validation
/reset- password	email, otp, password	token, newPassword	

عمره ما يعدي ال
validation

ليه ده بيحصل؟

لو فهمت التدفق هبقى واضح. الزبون بيعت JSON. الأول بيعدي على
: validate(schema)

Request Body



validate (بينصف المفتاح المطلوب)



controller (!بيقري مفتاح مختلف)

مثال /login : الزبون بيعت { "identifier": "a@b.com", "password": "123456" } ال schema بيطلب email – مافيش email في ال body، فال validation بيفشل بغلط 400. ولو الزبون بعث email عشان يعدي ال validation، ال controller بيدي identifier ، فيبقى undefined، وال service مش هيقدر يلاقي اليوزر.

4. مشكلة رقم 3 – طول ال OTP غير متطابق

ال OTP بيتم توليده في ملف ال utils:

```
// src/shared/utils/otp.ts
const generatedOtp = customAlphabet("0123456789", 5); // أرقام 5
```

بس ال validator بيقول:

```
otp: z.string().length(6) // ← 6 أرقام
```

الفرق بين 5 و6:

• لو عدّلنا مشكلة رقم 2، الـ OTP اللي هيوصلك في الإيميل (5 أرقام) **مش هيقدر يعدي الـ validation** لأن بيطلب 6.

• الـ 5 أرقام اللي بيظهر للإيميل والـ 6 أرقام المطلوب في الـ body — مش هيقبلوا بعض.

الحل: ثبّت الرقم. يا نخلي التوليد 6 أرقام، يا نخلي الـ 5 validation. أنضف تعملهم 6 عشان أعلى أمان.

5. مشكلة رقم 4 – وحدات الوقت غلط

دي مشكلة "ميلي سكوند vs سكوند". الـ JWT بياخد **السكوند**، وباقي الأدوات بتحسب بالـ **ميلي سكوند**. الـ env. فيه:

```
ACCESS_TOKEN_EXPIRY= 86400000 // المقصود: يوم واحد (بالميلي)
REFRESH_TOKEN_EXPIRY = 604800000 // المقصود: أسبوع (بالميلي)
```

اللي بيحصل فعليًا:

المكان	الكود	القيمة الفعلية	الصواب
token.service → JWT	<code>expiresIn: 86400000</code>	تقريبًا 1000 يوم!	86400 ثانية = يوم
الكوكي maxAge	<code>604800000 * 1000</code>	آلاف الأيام!	604800000 ميلي سكوند بس
توكن الـ OTP	<code>5 * 60 * 1000</code>		

300 = 5 * 60
ثانية

3.5 أيام بدل
5 دقائق!

النتيجة: التوكنات بتفضل "تسامح" فترة طويلة جدًا – ثغرة أمان.

6. مشكلة رقم 5 – اسم الفولدر Category

الفولدر معمول بحرف كبير:

```
src/modules/Category/ ← غلط  
src/modules/auth/      ← صح  
src/modules/brand/     ← صح  
src/modules/user/      ← صح
```

ليه مهم؟ لأن على لينكس و ال servers (اللي بتعمل deploy عليها) أسماء الملفات **case-sensitive**. لو كتبت في الاستيراد `category.routes` والفولدر اسمه `Category` ، هيعملهاش. الأفضل توحيد كل الموديولات بحرف صغير.

7. مشكلة رقم 6 – ملفات dist اتعملها commit

ملفات البناء `dist/` (نتيجة `npm run build`) متعملة **staged** في **git**، وده غلط لأن:

- مفيش حد محتاج ملفات البناء في ال `repo` — هي بتتولد من الكود.
- بتعمل تعارضات كثيرة في ال `merge` (لأنها بتتغير مع كل `build`).
- الحل: إضافة `/dist` في ملف `.gitignore`.

8. مشكلة رقم 7 – أمان ال OTP

في تدفق ال `sendOtp` (نسيان كلمة السر)، ال OTP بيتخزن **plaintext** في قاعدة البيانات:

```
authModel.updateOtp(user.id, generatedOtp, ...) // النص الصريح
```

بينما في تدفق الـ register سيتم hash قبل ما يتحفظ في الـ JWT. لو حد وصل للداتايز هيعرف كل الـ OTPs. الأفضل يكون hash في الحالتين.

9. ملاحظة صغيرة في الـ validate middleware

```
catch (error) {  
  if (error instanceof z.ZodError) {  
    return next(new AppError(...));  
  }  
  // نهائياً - الطلب "يعلق" next() مفيش -> Zod لو الخطأ مش  
}
```

لو خطأ من نوع ثاني غير الـ Zod، مفيش next() ولا next(error) بيتنادي، فالـ request بيوقف معلق. الحل: إضافة return next(error) في الـ catch.

10. ترتيب الإصلاح المقترح

الأولوية	المشكلة	فين
1	إصلاح الـ auth validators	auth.validators.ts
2	تنفيذ موديول Category	/src/modules/Category
3	توحيد طول الـ OTP	otp.ts + auth.validators.ts
4	إصلاح وحدات الوقت	token.service.ts + auth.service.ts
5	إعادة تسمية الفولدر Category	/src/modules
6	إضافة /dist للـ gitignore	gitignore.

الهدف: مشروع يشتغل من أول مرة من غير مفاجآت، والتاسك بتاعك (Category)
يكون منفذ على نفس نمط الموديولات الثانية.